

Notes

Software Quality Assurance

Lecture : 01

1. Software Quality

Software quality means how well a software works according to its requirements and how well it satisfies users.

Two types of requirements:

A. Functional Requirements — What the software does

- **Correctness:** It does the required tasks correctly.
- **Completeness:** All required features are included.

B. Non-Functional Requirements — How the software works

- **Reliability:** Works properly without frequent failures.
- **Performance:** Responds quickly and uses resources properly.
- **Usability:** Easy for users to understand and use.
- **Security:** Keeps data safe from hackers and unauthorized access.
- **Maintainability:** Easy for developers to fix and update

2. Software Quality Attributes

These are the **properties** that tell us how good a software system is.

Performance

How fast and efficiently the software works.

- **Speed:** How quickly it responds.
- **Efficiency:** How well it uses CPU, memory, and network.

Usability

How easy the software is for normal users.

- **Simplicity:** Easy to understand.
- **Accessibility:** Can be used by people with different abilities.

Reliability & Availability

The system should work correctly without crashes.

- **Consistency:** Gives expected results every time.
- **Uptime:** How much time the system is working and available.

Security

Protects software and user data.

- **Privacy:** User information is protected.
- **Integrity:** Data is not lost or damaged.

Maintainability

How easily developers can change or repair software.

- **Modifiability:** Easy to fix bugs or add features.
- **Testability:** Easy to test for errors.

Scalability

The software can handle **more users or more data** without becoming slow.

Portability

How easily software can be moved to **different devices, operating systems, or hardware**.

3. Quality Engineering (QE)

Quality Engineering means building quality into the software **from the beginning**, instead of finding problems only at the end.

Components:

- **Shift-Left Approach:** Moving testing activities to the left side of the project timeline. This means testing code as soon as it is written, rather than waiting for the end.
- **Continuous Integration and Continuous Deployment (CI/CD):** Testing and releasing software automatically in small, safe batches.
- **Automation First:** Use automated tests for repetitive tasks.
- **Whole-Team Ownership:** Everyone in the team is responsible for quality.

4. Quality Engineering Goals

The main goals are:

1. **Early Issue Detection** — Find problems early.
2. **Continuous Improvement** — Keep improving software and processes.
3. **Integration into Development** — Include quality checks in every development stage.
4. **Customer Satisfaction** — Make software that satisfies users.

5. How to Measure Software Quality

Code Quality

- **Code Coverage:** How much code is tested.
- **Cyclomatic Complexity:** How complicated the code is.
- **Code Churn:** How frequently code is changed.
- **Technical Debt:** Time needed to fix existing quality problems.

Reliability

- **Uptime:** How long the system stays available.
- **MTBF (Mean Time Between failure):** Average time between failures.
- **MTTR (Mean time to recovery):** Average time needed to recover from a failure.
- **Error Rate:** How often errors occur.

Performance

- **Response Time:** How quickly the system responds.

- **Throughput:** How many requests it can handle per second.
- **Resource Utilization:** How much CPU, memory, and storage it uses.
- **Scalability:** How well it works when the workload increases.

6. Challenges of Quality Engineering

Some common problems are:

- Different teams may use different quality standards.
- Old software may be difficult to test and improve.
- It can be difficult to show the business value of preventing bugs.
- Modern systems are complex because they use microservices, cloud systems, and third-party services.

Simple meanings:

- **Microservices:** Different functions of a website are divided into separate services.
- **Cloud infrastructure:** Software runs on cloud servers.
- **Third-party integration:** Software uses an external service, such as a payment service.

7. Key Roles in a QE Team

- **SDET (Software Development Engineering in test):** Developer who creates automated testing tools.
- **Performance Engineer:** Checks performance under heavy traffic.
- **Security Engineer:** Finds security weaknesses and protects the system.
- **Continuous Improvement Engineer:** Uses data to improve the development process.

8. Popular QE Tools

Purpose	Tools
Test Automation	Selenium, Cypress
CI/CD	Jenkins, GitHub Actions
API Testing	Postman
Performance Testing	JMeter

